UNIQ ID: E0124092 DATE: 22/09/26

Implement MC Prediction and On-Policy MC Control

```
pip install gymnasium pandas numpy
```

```
Requirement already satisfied: gymnasium in /usr/local/lib/python3.13/dist-
packages (1.3.0)
Requirement already satisfied: pandas in /usr/local/lib/python3.13/dist-
packages (2.2.3)
Requirement already satisfied: numpy in /usr/local/lib/python3.13/dist-
packages (2.1.3)
Requirement already satisfied: cloudpickle>=1.2.0 in
/usr/local/lib/python3.13/dist-packages (from gymnasium) (3.1.2)
Requirement already satisfied: typing-extensions>=4.3.0 in
/usr/local/lib/python3.13/dist-packages (from gymnasium) (4.16.0)
Requirement already satisfied: farama-notifications>=0.0.1 in
/usr/local/lib/python3.13/dist-packages (from gymnasium) (0.0.6)
Requirement already satisfied: python-dateutil>=2.8.2 in
/usr/local/lib/python3.13/dist-packages (from pandas) (2.9.0.post0)
Requirement already satisfied: pytz>=2020.1 in
/usr/local/lib/python3.13/dist-packages (from pandas) (2025.2)
Requirement already satisfied: tzdata>=2022.7 in
/usr/local/lib/python3.13/dist-packages (from pandas) (2026.3)
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.13/dist-
packages (from python-dateutil>=2.8.2->pandas) (1.17.0)
```

```
import collections
import gymnasium as gym
import numpy as np
import pandas as pd

# ==========================================
# PART 1: FIRST-VISIT MC PREDICTION
# ==========================================


def sample_policy(state):
    """Stick (0) if score is 20 or more; otherwise Hit (1)."""
    player_score, _, _ = state
    return 0 if player_score >= 20 else 1


def generate_episode_prediction(env, policy):
    """Generates an episode using the prediction policy."""
    episode = []
    state, _ = env.reset()

    while True:
        action = policy(state)
        next_state, reward, terminated, truncated, _ = env.step(action)
        episode.append((state, action, reward))
        if terminated or truncated:
            break
        state = next_state
```

```
    return episode


def first_visit_mc_prediction(env, policy, num_episodes=500000):
    """Estimates the state-value function V using First-Visit MC."""
    returns_sum = collections.defaultdict(float)
    returns_count = collections.defaultdict(float)
    V = collections.defaultdict(float)

    for i in range(num_episodes):
        episode = generate_episode_prediction(env, policy)
        states_in_episode = [x[0] for x in episode]

        G = 0
        for t in reversed(range(len(episode))):
            state, action, reward = episode[t]
            G += reward

            if state not in states_in_episode[:t]:
                returns_sum[state] += G
                returns_count[state] += 1
                V[state] = returns_sum[state] / returns_count[state]

    return V

# ========================================
# PART 2: ON-POLICY MC CONTROL
# ========================================

def get_epsilon_greedy_action(Q, state, nA, epsilon):
    """Selects action using epsilon-greedy strategy."""
    if np.random.rand() < epsilon:
        return np.random.choice(nA)
    else:
        return np.argmax(Q[state])


def generate_episode_control(env, Q, epsilon, nA):
    """Generates an episode using epsilon-greedy policy based on current Q-
values."""
    episode = []
    state, _ = env.reset()

    while True:
        action = get_epsilon_greedy_action(Q, state, nA, epsilon)
        next_state, reward, terminated, truncated, _ = env.step(action)
        episode.append((state, action, reward))
        if terminated or truncated:
            break
        state = next_state

    return episode
```

```python
def mc_on_policy_control(env, num_episodes=50000, epsilon=0.1):
    """Optimizes the action-value function Q using On-Policy MC Control."""
    nA = env.action_space.n
    Q = collections.defaultdict(lambda: np.zeros(nA))
    returns_sum = collections.defaultdict(float)
    returns_count = collections.defaultdict(float)

    for i in range(num_episodes):
        episode = generate_episode_control(env, Q, epsilon, nA)
        state_action_in_episode = [(x[0], x[1]) for x in episode]

        G = 0
        for t in reversed(range(len(episode))):
            state, action, reward = episode[t]
            G += reward
            if (state, action) not in state_action_in_episode[:t]:
                returns_sum[(state, action)] += G
                returns_count[(state, action)] += 1
                Q[state][action] = (
                    returns_sum[(state, action)] / returns_count[(state,
action)]
                )

    return Q


# ========================================
# EXECUTION & RESULTS DISPLAY
# ========================================

if __name__ == "__main__":
    env = gym.make("Blackjack-v1", sab=True)

    print("--- Running First-Visit MC Prediction (500,000 episodes) ---")
    V = first_visit_mc_prediction(env, sample_policy, num_episodes=500000)

    print("Sample State Values:")
    sample_states = [(20, 10, False), (21, 10, False), (14, 5, True)]
    for s in sample_states:
        print(f"State {s}: Value = {V[s]:.4f}")

    print("\n--- Running On-Policy MC Control (50,000 episodes) ---")
    Q = mc_on_policy_control(env, num_episodes=50000, epsilon=0.1)

    q_dict_for_df = []
    for state, actions in sorted(Q.items()):
        q_dict_for_df.append(
            {
                "Player Score": state[0],
                "Dealer Card": state[1],
                "Usable Ace": state[2],
                "Q(Stick)": actions[0],
                "Q(Hit)": actions[1],
                "Best Action": "Stick" if np.argmax(actions) == 0 else "Hit",
            }
```

```
        )

    df = pd.DataFrame(q_dict_for_df)
    print("\nFirst few rows of optimized Q-values table:")
    print(df.head(15))

    env.close()
```

```
--- Running First-Visit MC Prediction (500,000 episodes) ---
Sample State Values:
State (20, 10, False): Value = 0.4363
State (21, 10, False): Value = 0.8903
State (14, 5, True): Value = -0.2414

--- Running On-Policy MC Control (50,000 episodes) ---

First few rows of optimized Q-values table:
    Player Score  Dealer Card  Usable Ace  Q(Stick)    Q(Hit) Best Action
0              4            1           0 -0.857143 -1.000000       Stick
1              4            2           0 -0.083333 -1.000000       Stick
2              4            3           0 -1.000000  0.050000         Hit
3              4            4           0 -0.500000  0.333333         Hit
4              4            5           0 -0.272727 -0.111111         Hit
5              4            6           0 -0.333333 -1.000000       Stick
6              4            7           0 -0.818182 -0.222222         Hit
7              4            8           0 -1.000000 -0.240000         Hit
8              4            9           0 -1.000000 -0.142857         Hit
9              4           10           0 -0.657143  0.000000         Hit
10             5            1           0 -0.666667 -0.523810         Hit
11             5            2           0 -0.300000 -0.200000         Hit
12             5            3           0 -0.028571 -1.000000       Stick
13             5            4           0 -0.391304 -1.000000       Stick
14             5            5           0 -0.027027 -1.000000       Stick
```